



SALEM Elie : Certificat de spécialisation Analyste de données massives
EYMA Cyril : Ingénieur diplômé de l'école d'ingénieurs du Cnam spécialité informatique, parcours
Systèmes d'information (ISI)

Projet NFE204

Bases de données documentaires et distribuées

Encadrant : Dr Philippe RIGAUX

Espace vert interactif



Table De Matière

Introduction.....	3
I- Présentation des Outils.....	4
A- Présentation de Hadoop.....	4
B- Présentation de Hive.....	5
C- Installation de Hadoop – Hive :	6
D- Installation de Hive:	11
II- Les Données.....	14
Le Projet et la récolte des données :	14
A- Mairie De Paris.....	14
B- Site web meslieux.paris.fr/parcs-et-jardins.....	14
C- TWITTER.....	16
D- Open weather map.....	17
III- HIVEQL.....	18
A- Créer une DATABASE.....	18
1- Création de la table de TWITTER.....	21
2- Création de la table Mairie de Paris.....	21
3- Création de la table Jardin de Paris.....	21
B- Fonction simple.....	22
C- Partitionnement	23
D- Multiple Inserts.....	24
E- Les Buckets :	25
F- Streaming :	26
G- Jointure :	28
1-JOIN : Intersection de deux tables.....	28
2-LEFT, RIGHT, FULL [OUTER] JOIN.....	28
3-LEFT SEMI-JOIN.....	28
4-Produit cartésiens ou cross-join.....	28
5-Map-side Joins.....	28
Statistiques et Data Mining (text mining).....	30
H- Explode :	31
I- Sentence et Ngrams.....	32
J- Context ngrams.....	32
Conclusion.....	34
Bibliographie.....	35
ANNEX 1.....	36
ANNEX 2.....	37
ANNEX 3.....	38
ANNEX 4.....	39

Chapitre 1

Introduction

Espace Vert interactif est un projet qui a pour but d'offrir un service de renseignement interactif sur les activités qu'il est possible de pratiquer dans les différents parcs et jardins de la ville de Paris. Ce service est imaginé sous la forme d'un site internet qui permet de rechercher et de filtrer les activités souhaitées. Par exemple : Je cherche un parc ou un jardin offrant un service de type « jeux pour enfant » ou « table de ping-pong » ou bien « fête foraine ». Selon les services sélectionnés par les soins de l'utilisateur, le site internet affiche un plan qui répertorie les parcs et jardins offrant ces services.

Ce site internet constitue l'objectif final de notre recherche. Cependant, ce rapport présente uniquement les premières pistes de recherche que nous avons investies pour commencer ce grand projet.

Nous allons dans un premier temps vous présenter les outils Hadoop/Apache Hive ainsi que les modalités d'installation de ces deux logiciels sur une plate-forme Linux.

Le système Hadoop est une solution qui s'est démocratisée depuis quelques années grâce au principe du Big Data. L'une des définitions du Big Data serait la récolte de toutes les données sur laquelle il est intéressant d'appliquer un traitement par rapport aux ressources dont l'on dispose. C'est dans cet esprit que nous allons vous présenter Hive, un outil de gestion de haut niveau des données massives.

Dans un deuxième temps, nous verrons comment importer les données de sources diverses : extraction des données de sites web, construction de fichiers JSON avec des flux de données tweetées, exploitation de nouvelles sources de données ouvertes.

Enfin nous présenterons un ensemble de requêtes HiveQL en mettant en avant leur performance dans un système distribué. Tout cela s'inscrivant dans notre projet de répertorier l'ensemble des espaces verts (parcs, jardins et squares) de la ville de Paris et en particulier des services libres d'accès et disponibles dans ces espaces.

Chapitre 2

I- Présentation des Outils

A- Présentation de Hadoop

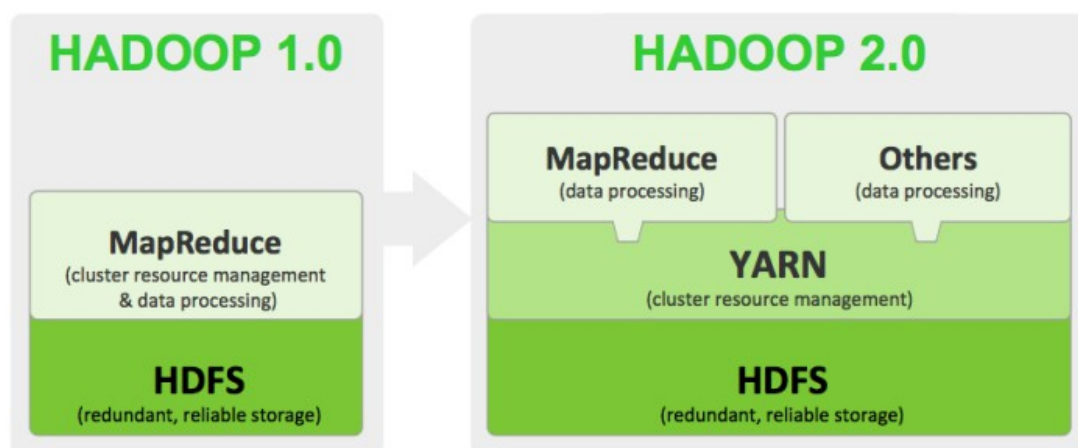
Hadoop est un framework open source de stockage et de traitement distribué. Il permet aux applications de travailler avec des milliers de nœuds et un grand volume de données selon un modèle de programmation simple. Hadoop a été inspiré par les publications MapReduce, GoogleFS et BigTable de Google. Il est conçu de manière scalable. Il permet donc de passer d'une architecture de serveur unique à des milliers en répartissant la charge. Plutôt que de s'assurer une haute disponibilité par la qualité de matériel, celui-ci est géré de manière logicielle. Le produit permet donc de détecter et de gérer les pannes (Failover).

Le stockage distribué est assuré par Hadoop distributed file system (HDFS). Ce système de fichiers a été conçu de manière à opérer sur un cluster de serveurs « low- cost ». Du fait de la redondance de données, qu'il permet de gérer de manière transparente, il permet également une tolérance aux pannes. Le traitement distribué de ces mêmes fichiers HDFS (et sur les mêmes serveurs où ils sont stockés) est quant à lui géré dans le framework Mapreduce. Ce dernier est caractérisé par un modèle de programmation conçu pour paralléliser les programmes Hadoop et gérer de manière transparente le mode distribué.

Dans la version 1 de Hadoop, Mapreduce jouait deux rôles : distribuer les traitements, et gérer les ressources. Dans la version 2 de Hadoop, Mapreduce se limite à son premier rôle.

Le second est désormais géré par YARN (Yet Another Resource Manager). Le gros avantage de ce découplage est d'avoir la possibilité de se passer, quand il le faut, des contraintes de latence de Mapreduce (conçu plutôt pour un traitement en batch), et de pouvoir utiliser, selon le type du besoin à couvrir, d'autres frameworks de traitement distribués directement au-dessus de HDFS/Yarn.

Figure : Evolution de HADOOP 2 par rapport à Hadoop 1



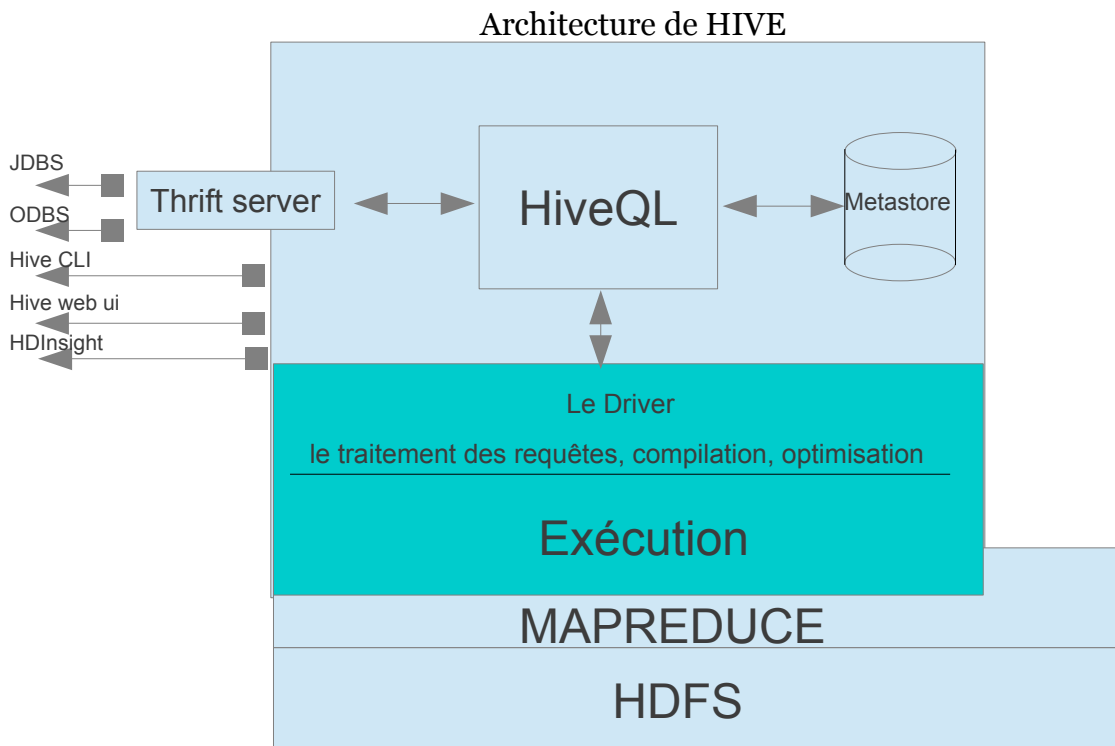
Le principe de Hadoop sur son infrastructure Maître-esclave étant démontré dans le cours, nous ne nous attarderons pas sur ce point durant le rapport. Il est cependant important de noter que Hive se base sur cette infrastructure pour le traitement de données.

B- Présentation de Hive

Hive est un membre important de l'écosystème Hadoop. Il est intégré au-dessus de Hadoop pour fournir un résumé de données, de recherche et d'analyse. Hive utilise le langage de type SQL pour intégrer les données depuis HDFS. Celui-ci est simplifié par rapport à l'utilisation de Mapreduce, qui utilise la programmation de type Java. L'accès aux données se fait via des tables structurées. Il se matérialise par la création d'un plan d'exécution de la requête SQL qui se traduit par la création et l'exécution d'un job MapReduce. Pour accélérer les requêtes, il fournit des index, y compris les index bitmap.

Hive a été développé par Facebook dans une vision d'utilisation de SQL pour traiter les données Big Data dans le but de réduire la difficulté de la programmation. Le produit est devenu open source depuis août 2008.

Les requêtes Hives sont faciles à appréhender pour les personnes sans connaissance particulière en langages de programmation. Les requêtes envoyées dans Hive sont traduites en mapreduce pour être exécutées.



Hive fonctionne de deux manières : en mode local ou bien en mode partagé.

En mode local, les requêtes sont lancées dans un processus unique et utilisent le système de fichier local. En mode partagé, nous sommes dans une exécution d'un traitement distribué par MapReduce. Les mappers et réducteurs lancent les différents processus et utilisent le système de fichiers distribués Hadoop.

Hive stocke le contenu des données dans HDFS et les métadonnées dans Metastore. Dans le Metastore, on retrouve les définitions des tables, leur emplacement, les informations sur leurs données, leurs partitionnement, les buckets, les clusters, etc.

Par défaut, les métadonnées sont stockées dans la base de données Apache Derby intégrée, mais cela ne peut être utilisé dans une configuration multi-utilisateurs. Apache Derby est un moteur de base de données relationnelles écrit en langage Java qui peut être embarqué dans des programmes écrits en Java. Il est multi plate-forme et de très petite taille (2MB). Il s'intègre particulièrement bien dans toute application Java.

Dans un environnement multi-utilisateurs, nous pouvons utiliser des bases de données comme mysql, postgresql, oracle pour les stocker. Le driver traite les requêtes hiveQL, les compile, puis les optimise pour les transformer en un ou plusieurs MAPREDUCE JOB.

Les informations sont stockées dans HDFS et elles sont enregistrées dans un répertoire appelé « entrepôt de stockage » Hive qui est défini par la variable de propriétés *hive.metastore.warehouse.dir*. Par défaut, il utilisera le répertoire /user/hive/warehouse (**hdfs://localhost:9000/user/hive/warehouse/**).

Nous pouvons lancer des requêtes dans Hive en ligne de commande (hive CLI) ou bien par une architecture client-serveur. Ces requêtes peuvent être écrites en différentes langues.

Actuellement, il y a quatre formats de fichiers pris en charge dans Hive :

- TEXTFILE,
- SEQUENCEFILE, un fichier binaire plus compact que Textfile et plus performant (on pourra l'utiliser aussi pour exporter les données),
- Rfile, un format qui a été développé par Facebook, rapide dans les chargements de données, aussi bien pour les traitements des requêtes que pour l'utilisation de l'espace de stockage, et très adaptable à des modèles de charge de travail dynamique,
- Orcfile, l'optimisation du format Rfile.

Pour résumé, voici les caractéristiques principales de Hive :

- 1- Indexation pour prévoir l'accélération, de types compactage ou d'indice de Bitmap 0.10 , plusieurs types d'index sont prévus dans le futur.
- 2-Types de stockage différents, tels que du texte brut, rcfile , HBase , ORC , et d'autres.
- 3- Stockage des métadonnées dans un SGBDR, réduisant considérablement le temps d'effectuer des contrôles sémantiques lors de l'exécution de la requête.
- 4-Exploitation de données compressées stockées dans Hadoop écosystème, y compris l'algorithme gzip , bzip2 , accrocheur , etc.
- 5- Des fonctions définies par l'utilisateur ou intégrées pour manipuler les dates, les chaînes de caractères et autres outils d'exploration de données.
- 6- Requetes de type SQL (HiveQL), qui sont implicitement converties en MapReduce.

Afin de vous donner une meilleure idée du produit, nous allons vous présenter son installation puis les différentes utilisations que nous avons testées.

C- Installation de Hadoop – Hive :

Il y a plusieurs moyens d'installer HIVE.

Un des moyens le plus simple à des fins d'évaluation c'est d'utiliser une machine virtuelle fournie par l'une des distributions Hadoop. Les plus connues sont :

1. Cloudera : <http://www.cloudera.com/>
2. Hortonworks : <http://hortonworks.com/>
3. MapR Technologies : <http://www.mapr.com/>

Le deuxième moyen est d'installer apache Hive via ce site <https://hive.apache.org/>. Mais pour cela il faut installer Hadoop auparavant et le lancer. On a utilisé l'interface graphique <http://hortonworks.com/> pour la comparaison avec avec l'interface CLI.

Dans un premier temps, nous allons vous expliquer comment installer Hadoop en local pour pouvoir par la suite utiliser Hive.

Installation de Hadoop environment initial :

Ubuntu 14.04, Hadoop 2.6.0, OpenJDK 1.7

I. Installation du JAVA JDK

Connexion en tant que root :

```
sudo -s
```

Installation de Java Runtime :

```
sudo apt-get install default-jre
```

Installation du Java JDK (OpenJDK v 1.7 or newer) :

```
sudo apt-get update
```

```
sudo apt-get install default-jdk
```

Vérification de la version Java :

```
Java --version
```

II. Installation du SSH et RSYNC

Installation (SSH and RSYNC) :

```
sudo apt-get install ssh
```

```
sudo apt-get install rsync
```

Créer et configurer les certificats SSH (Setup passphraseless ssh)

Pour activer les connexions sans mot de passe il faut générer une clé SSH avec une passphrase vide.

Utiliser l'utilisateur HDUSER:

Hadoop a besoin d'une connexion ssh pour manager le serveur. Il faut générer une clé SSH avec l'utilisateur hduser qu'il faut créer. La clé RSA est générée sans passphrase dans le répertoire ~/.ssh.

```
su - hduser  
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa  
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys  
ssh localhost
```

Activer l'accès SSH sur la machine locale avec la nouvelle clé créée. Tester la configuration SSH en se connectant en local sur la machine en prenant l'utilisateur « hduser ». Cela va sauvegarder en local l'identification ssh du serveur avec l'utilisateur « hduser » dans le fichier ~/.ssh/known_hosts.

III. Récupération et installation Hadoop.

Récupération Hadoop (Version Stable) :

```
wget http://apache.tradebit.com/pub/hadoop/common/current/hadoop-2.6.0.tar.gz
```

Extraction du fichier :

```
tar xfz hadoop-2.6.0.tar.gz
```

Déplacement en local :

```
mv hadoop-2.6.0 /usr/local/hadoop
```

IV. Éditer les fichiers de configuration Hadoop.

Récupération du chemin JAVA :

```
update-alternatives --config java
```

```
/usr/lib/jvm/java-7-openjdk-i386 /jre/bin/java
```

Modification des fichiers suivant :

1. ~/.bashrc
2. /usr/local/hadoop/etc/hadoop/hadoop-env.sh
3. /usr/local/hadoop/etc/hadoop/core-site.xml
4. /usr/local/hadoop/etc/hadoop/yarn-site.xml
5. /usr/local/hadoop/etc/hadoop/mapred-site.xml
6. /usr/local/hadoop/etc/hadoop/hdfs-site.xml

(1)~/.bashrc

```
nano ~/.bashrc
```

Ajouter à la fin du fichier les variables d'environnement JAVA_HOME et HADOOP_INSTALL. Changer JAVA_HOME avec le chemin correct qui a été récupéré auparavant, et HADOOP_INSTALL du répertoire d'installation.

```
#HADOOP VARIABLES START
export JAVA_HOME=/usr/lib/jvm/java-7-openjdk-i386
export HADOOP_INSTALL=/usr/local/hadoop
export PATH=$PATH:$HADOOP_INSTALL/bin
export PATH=$PATH:$HADOOP_INSTALL/sbin
export HADOOP_MAPRED_HOME=$HADOOP_INSTALL
export HADOOP_COMMON_HOME=$HADOOP_INSTALL
export HADOOP_HDFS_HOME=$HADOOP_INSTALL
export YARN_HOME=$HADOOP_INSTALL
export
HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_INSTALL/lib/native
export HADOOP_OPTS="-Djava.library.path=$HADOOP_INSTALL/lib"
#HADOOP VARIABLES END
```

Afin de mettre à jour les variables :

```
source ~/.bashrc
```

(2) /usr/local/hadoop/etc/hadoop/hadoop-env.sh :

```
nano /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

Mettre à jour JAVA_HOME pour indiquer à Hadoop notre librairie Java :

```
JAVA_HOME=/usr/lib/jvm/java-7-openjdk-i386
```

(3) /usr/local/hadoop/etc/hadoop/core-site.xml

```
nano /usr/local/hadoop/etc/hadoop/core-site.xml
```

Ajouter les informations suivantes au fichier de configuration, ici on définit l'emplacement de HDFS sur notre serveur local :

```
<property>
  <name>fs.default.name</name>
  <value>hdfs://localhost:9000</value>
</property>
```

(4) /usr/local/hadoop/etc/hadoop/yarn-site.xml :

```
nano /usr/local/hadoop/etc/hadoop/yarn-site.xml
```

Ajouter les informations suivantes au fichier pour configurer YARN qui gère les ressources :

```
<property>
  <name>yarn.nodemanager.aux-services</name>
  <value>mapreduce_shuffle</value>
</property>

<property>
  <name>yarn.nodemanager.aux-
services.mapreduce.shuffle.class</name>
  <value>org.apache.hadoop.mapred.ShuffleHandler</value>
</property>
```

(5) /usr/local/hadoop/etc/hadoop/mapred-site.xml

Copier le template du fichier mapred-site.xml.template :

```
cp /usr/local/hadoop/etc/hadoop/mapred-site.xml.template
   /usr/local/hadoop/etc/hadoop/mapred-site.xml
```

Configuration de MAPREDUCE:

```
nano /usr/local/hadoop/etc/hadoop/mapred-site.xml
```

Ajouter les informations suivantes au fichier pour le configurer :

```
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
</property>
```

(6)/usr/local/hadoop/etc/hadoop/hdfs-site.xml

Créer 2 répertoires pour le stockage Hadoop :

```
mkdir -p /usr/local/hadoop_store/hdfs/namenode
mkdir -p /usr/local/hadoop_store/hdfs/datanode
```

Modifier le fichier :

```
nano /usr/local/hadoop/etc/hadoop/hdfs-site.xml
```

Nous définissons la réplication des données à 1 puis nous configurons le chemin du NameNode et du Datanode.

```
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>
<property>
  <name>dfs.namenode.name.dir</name>
  <value>file:/usr/local/hadoop_store/hdfs/namenode</value>
</property>
<property>
  <name>dfs.datanode.data.dir</name>
  <value>file:/usr/local/hadoop_store/hdfs/datanode</value>
</property>
```

V. Changer les droits des répertoires.

Remplacer l'utilisateur local par l'utilisateur Hduser.

```
sudo chown hduser:hadoop -R /usr/local/hadoop
sudo chown hduser:hadoop -R /usr/local/hadoop_store
```

VI. Formater le nouveau système de fichier Hadoop.

Avec l'utilisateur Hduser (pas avec le super utilisateur "root") :

```
hdfs namenode -format
```

VII. Démarrer Hadoop

Avec l'utilisateur Hduser (pas avec le super utilisateur "root") :

```
start-dfs.sh
```

```
start-yarn.sh
```

OU lancer tous les services :

```
start-all.sh
```

VIII. Interface Web

ResourceManager @- <http://localhost:8088/>

NameNode @- <http://localhost:50070/>

D- Installation de Hive:

Télécharger une version stable de Apache Hive :

<http://mirrors.ircam.fr/pub/apache/hive/stable/apache-hive-0.14.0-bin.tar.gz>

Dézipper l'archive :

```
tar -zxvf apache-hive-0.14.0-bin.tar.gz
```

Placer apache hive dans /usr/local/

```
mv apache-hive-0.14.0-bin /usr/local/hive
```

Changer le groupe et l'utilisateur de tous les fichiers et dossiers de Hive.

```
sudo chown hduser1:hadoop -R /usr/local/hive
```

Editer le fichier .bashrc pour ajouter les liens vers hive/bin

```
nano ~/.bashrc
```

```
export HIVE_HOME=/usr/local/hive
export PATH=$PATH:$HIVE_HOME/bin
```

Afin de mettre à jour les variables :

```
source ~/.bashrc
```

Rentrer dans le dossier hive puis démarrer avec :

```
bin/hive
```

Il faut que Hadoop soit démarré au préalable sinon ça ne fonctionne pas.

Pour ce faire :

```
cd hadoop
sbin/hadoop-start.all
```

Pour vérifier que tout fonctionne :

Lancer la commande :

```
jps
```

La sortie:

```
5287 DataNode
```

```
5127 NameNode
```

```
5484 SecondaryNameNode
```

```
8716 Jps
```

```
4654 NodeManager
```

```
6094 ResourceManager
```

Hadoop requetage. On rentre dans le dossier Hadoop dans usr/local/hadoop.

Pour lister les fichiers on utilise la fonction « ls » comme sur un système unix.

```
bin/hadoop fs -ls /
```

Pour créer un fichier dans Hadoop on utilise la fonction « mkdir » le système unix.

```
bin/hadoop fs -mkdir /jardin
```

```
bin/hadoop fs -mkdir /mairie
```

Pour ajouter un fichier dans Hadoop HDFS on utilise la fonction « PUT ».

```
bin/hadoop fs -put
```

```
/home/hduser1/base_projet/data_tweet/twitDB2.json /jardin
```

```
bin/hadoop fs -put
```

```
/home/hduser1/base_projet/data_tweet/twitDB.json /jardin
```

```
bin/hadoop fs -put
```

```
/home/hduser1/base_projet/parcsetjardinsparis2010.csv /mairie
```

Pour copier un fichier dans autre dossier « test » par exemple on va utiliser la fonction «cp ».

```
bin/hadoop fs -cp /jardin/twitDB2.json /test
```

Sur le lien <http://localhost:50070/explorer.html> nous pouvons voir tous les dossiers et les fichiers que nous avons créés.

Pour supprimer un fichier on utilise la fonction « rm ».

```
bin/hadoop fs -rm /jardin/twitDB.json
```

Cependant Hadoop a un système de poubelle qui a pour but de restaurer un fichier ou un dossier.

Il faut ajouter la commande -skipTrash afin de supprimer un fichier définitivement. Pour supprimer un dossier, il faut utiliser la fonction récursive « rmr ».

```
bin/hadoop fs -rmr -skipTrash /test
```

On peut aussi utiliser les fonctions « cat » et « less » sur les fichiers pour les visionner sur

l'écran.

```
bin/hadoop fs -cat /jardin/twitDB.json | less
```

(pour sortir de « less » il faut utiliser la touche « q » du clavier)

Faire un Word count avec Hadoop.

On crée deux fichiers dans le dossier /Hadoop.

```
echo "bla bla" > a.txt
echo "bla wa wa " > b.txt
```

On ajoute les fichiers dans un dossier sur HDFS avec la fonction “put”.

```
bin/hadoop fs -put /test/a.txt /test/b.txt
```

Puis on lance le scripte JAVA pour word count, le dossier out permettra de récupérer la sortie du word count.

```
bin/hadoop jar share/hadoop/mapreduce/hadoop-mapreduce-
examples-2.6.0.jar wordcount /test /out
```

```
bin/hadoop fs -ls /out/*
part-r-00000
```

On imprime le résultat du word count avec la fonction « cat ».

```
bin/hadoop fs -cat /out/*
bla 3
wa 2
```

Nous pourrions également faire la même chose avec nos deux fichiers de twitter!

```
bin/hadoop jar share/hadoop/mapreduce/hadoop-mapreduce-
examples-2.6.0.jar wordcount /jardin /jardin-out
```

Chapitre 3

II- Les Données

Le Projet et la récolte des données :

On a récolté quatre bases de données aux sources suivantes :

A- Mairie De Paris

<http://opendata.paris.fr/explore/dataset/parcsetjardinsparis2010/?tab=table> « Les espaces verts », en format CSV.

Ce document représente les couches surfaciques qui localisent géographiquement l'emprise des espaces verts publics (parcs, squares, promenades, etc.) gérés par la Mairie de Paris. (~5,9 Mo)

Dans cette base il y a 1040 lignes. Chaque ligne représente un espace vert. Les champs principaux sont : le nom, les coordonnées polygone, adresse, année d'ouverture, année de rénovation, numéro, arrondissement, statut (ouvert ou fermé), et la surface en mètres carrés.

B- Site web meslieux.paris.fr/parcs-et-jardins

<http://meslieux.paris.fr/parcs-et-jardins> Sur ce site internet, on trouve 497 liens renvoyant à différents espaces verts dont 118 ayant strictement un nom, des services associés, l'adresse, l'historique et les horaires d'ouverture de ces espaces verts.

Pour extraire ces données, on a utilisé SCRAPY, un framework open-source permettant la création de robots d'indexation développés en Python et qui utilisent le langage XPATH pour extraire les données en format Json.

Pour l'installation on utilise la commande pip : `pip install Scrapy`

Pour créer notre projet : `scrapy startproject jardin`

On obtient un dossier qui s'appellera jardin dans ce dossier on retrouve :

- `scrapy.cfg` : configuration du projet.
- Jardin/ : Un autre dossier jardin où on écrit notre code pour configurer notre spider.
 - `jardin/items.py` : Dans ce fichier on crée les items que l'on veut récupérer de nos sites, exemple: les services.
 - `jardin/pipelines.py` : Dans ce fichier on configure la manière de récupérer les données.
 - `jardin/settings.py` : Dans ce fichier on règle les configurations du spider.
 - `jardin/spiders/` : Dans ce dossier on ajoute notre spider.

Pour construire notre spider on a suivi le premier tutoriel du site <http://doc.scrapy.org/en/0.24/intro/tutorial.html>

1- On a défini nos 6 items dans le fichier item.py :

```
from scrapy.item import Item, Field
class Website(Item):
    nom = Field()
    link = Field()
    nb_service = Field()
    Adresse=Field()
    histoire=Field()
    service = Field()
```

2- pipelines.py: Ici on définit l'encodage des données en utf8. Chaque item représente un site internet. On construit ligne par ligne note fichier json pour faciliter la dé-sérialisation avec HIVE.

```
import json
import codecs
class JardinPipeline(object):
    def __init__(self):
        self.file = codecs.open('data_utf8.json', 'w', encoding='utf-8')
    def process_item(self, item, spider):
        line = json.dumps(dict(item), ensure_ascii=False) + "\n"
        self.file.write(line)
        return item
    def spider_closed(self, spider):
        self.file.close()
```

3- Voici le code de notre spider.

```
from scrapy.spider import Spider
from scrapy.selector import Selector
from scrapy.contrib.spiders import CrawlSpider, Rule
from scrapy.contrib.linkextractors.sgml import SgmlLinkExtractor
from scrapy.utils.response import get_base_url
import re
from jardin.items import Website
from scrapy.contrib.linkextractors import LinkExtractor
class DmozSpider(Spider):

    name = "jardin"
    allowed_domains = ["http://equipement.paris.fr"]
    """
    il manque les 497 liens des sites internet
    """
    start_urls = [ 'http://equipement.paris.fr/jardin-des-tuileries-1795',... ]

    def parse(self, response):
        """
        Ici on utilise xpath pour récupérer les données.
        Les Services et les nombres des services sont des tableaux.
        Historique, adresses, ... sont des variables simples.
        """
        sel = Selector(response)
        site = sel.xpath('/html/body/div/div/div/div/div/div/ul/li') # on définit un path
```

```

items = []
item = Website()
item['service']=site.xpath('img/@title').extract()
item['nb_service']=site.xpath('div/text()').extract()
for site in site :
    base_url = get_base_url(response)
    item['link']= base_url
    item['Adresse']=map(unicode.strip,site.xpath('//div[@itemprop="address"]//text()
[normalize-space()]).extract())
    item['nom'] = map(unicode.strip,site.xpath('/html/body/div/div/div/div/h2/text()').extract())
    item['histoire'] = site.xpath('//div[@id ="descriptionTxt"]/descendant::text()').extract()
    items.append(item)
return item

```

Une autre méthode est possible pour crawler la totalité des sites sans avoir à mettre les 497 liens. Il faut personnaliser le spider pour qu'il explore tous les liens de la page principale avec une profondeur égale à un.

On remarque déjà qu'il y a une grande différence entre la base construite à l'aide de Scrapy et celle fournie par la Mairie de paris. Une première interrogation est de savoir comment joindre ces deux bases. Cette partie sera traitée à l'aide de HIVE.

(Un début de travail est déjà fait hors du cadre de votre cours qui peut aider à visualiser les données sur une carte à l'aide de D3.js. Voici le lien : <http://www.opttop.atelier-architecture-situee.fr/map/index3.html>. Les espaces verts (1040) sont représentés par des points rouges et un texte marque les services présents dans ces espaces sachant que nous n'avons pu en récupérer que 118).

C- TWITTER

API twitter <https://apps.twitter.com>

A l'aide du module **tweepy** développé en python on a pu récupérer le flux des données de twitter.

- 1- On crée un compte twitter pour les développeurs
- 2- On installe tweepy a l'aide du git

```

git clone https://github.com/tweepy/tweepy.git
cd tweepy
python setup.py install

```

Dans ce fichier il y a un dossier exemple que l'on modifie.

On ajoute nos identifiants de tweeter puis à la fin du fichier on ajoutera nos filtres (`stream.filter(track=['jardin paris', 'parc paris','espace vert paris', 'Paris square'])`). Voici le lien du fichier <https://github.com/tweepy/tweepy/blob/master/examples/streaming.py>

On applique les filtres suivants : « Paris jardin », « Paris Parc », « espace vert paris » et « Paris squares ». On récupère un fichier Json qui contient les tweets associés aux espaces verts en temps réel. (~4 Mo avec 800 tweets par semaine ~= 208 Mo avec 41 600 tweets par an).

On a déjà plus de 45 Mb de données. Cette base de données est constituée de 9407 lignes pour le moment. Dans chaque tweet il y a 160 variables que l'on pourra décomposer en 7 grandes parties.

Tweet , user , geo , coordinates , place , entities , retweeted_status. Voci un lien qui décrit toutes les variables <https://dev.twitter.com/overview/api/tweets>.

Nous avons décomposé les tweets grâce au logiciel de la boite DATAIKU <http://www.dataiku.com/> et nous avons conclu que chaque tweets est composé de variables inégales.

Avec 45 MB de données on est loin de vouloir passer en distribué pour faire nos requêtes.

Nous avons pensé à une autre méthode que Tweepy pour récolter les tweets afin de les ajouter directement dans HDFS. On voulait utiliser Flume Apache, mais malheureusement cette partie est encore en mode expérimental, le filtre ne fonctionne pas.
(<https://flume.apache.org/FlumeUserGuide.html>)

Une autre méthode pourrait être envisagée à l'aide des modules Hadoopy.
(<http://www.hadoopy.com/en/latest/>)

D- Open weather map

<http://openweathermap.org/> permettra d'ajouter la météo en temps réel en format Json.

On pourra inclure cette table plus tard pour des besoins d'analyse de données (il serait intéressant d'étudier l'influence des données météorologiques sur les événements).

III- HIVEQL

A- Créer une DATABASE

Hive supporte différents types de données, les complexes et les primitives. Elles sont basées sur le langage JAVA. Les primitives sont les suivantes : TINYINT, SMALLINT, INT, BIGINT, FLOAT, DECIMAL, DOUBLE, BOOLEAN (false et true), STRING, BINARY, TIMESTAMP, DATE.

Les complexes sont les suivantes : Array, Map, Struct.

Cela nous permet de créer des tables plus complexes que mysql.

La création de table doit être exactement identique à la structure des variables que l'on va injecter dedans.

Exemple de table :

```
CREATE TABLE employees (  
name STRING,  
salary FLOAT,  
subordinates ARRAY<STRING>,  
deductions MAP<STRING, FLOAT>,  
address STRUCT<street:STRING, city:STRING, state:STRING, zip:INT>);
```

Pour changer un type de données, on utilise la fonction `CAST`.

Exemple : `CAST ( '666' , INT)`

On commence par créer notre DATABASE « jardins » dans hive dans laquelle on va ajouter nos tables.

DATABASE dans Hive permet de créer un dossier dans le Metastore. Pour organiser les tables, on peut notamment :

- Ajouter des commentaires avec la fonction `COMMENT`
- Choisir l'emplacement de notre database avec la fonction `location '/ou/je/veux/'`
- Supprimer avec la fonction `drop`
- Décrire avec la fonction `describe database nom_table`.

Si on ne spécifie pas la DATABASE au début, on va enregistrer nos tables dans la DATABASE default. La fonction `USE nom_database` nous permet d'utiliser la database que nous avons créée au préalable. Il y a plusieurs fonctions comme `show database`, `show database like 'j*'`, `show database rlike '*n'` pour nous aider à chercher les databases. Ces fonctions s'appliquent aussi bien aux tables.

```
CREATE DATABASE jardin  
COMMENT 'Notre base principale' ;
```

```
describe database jardin;
```

```
jardin      Notre base principale
```

```
hdfs://localhost:9000/user/hive/warehouse/jardin.db    hduser1
USER
```

```
Time taken: 0.027 seconds, Fetched: 1 row(s)
```

```
USE jardin;
```

Une option pratique que l'on peut ajouter est

'**set hive.cli.print.current.db=true**' qui permet de savoir dans quelle table on travaille.

Exemple : **hive (jardin)>** , le prompt de Hive indique le nom de notre database. Pour supprimer cette fonction, il faut ajouter la même ligne de commande avec false à la fin. (**set hive.cli.print.current.db=false;**).

Afin d'effectuer le traitement des données de tweets, nous avons créé une table très simple avec une seule colonne qui va contenir toutes les données JSON, car chaque ligne de notre base représente un tweet. La fonction **IF NOT EXISTS** de Hive nous permet de créer la table si elle n'existe pas déjà.

```
CREATE TABLE IF NOT EXISTS jardin_service
(jardin_json STRING);
```

Avant d'effectuer un chargement de données, il est bon de nettoyer la table si celle-ci a déjà été chargée.

```
TRUNCATE TABLE tweet_api;
```

Les données n'étant pas normalisées, nous allons tenter de les travailler de cette manière. Nous allons les charger avec la fonction **LOAD**. L'idée, ici est de pouvoir simplement insérer les données de tweets ligne par ligne.

```
LOAD DATA INPATH '/jardin' OVERWRITE INTO TABLE tweet_api;
```

Il y a deux moyens de créer les tables : Managed Tables et External Tables.

La première **locally maged table** : Hive copie les données dans le METASTORE **/user/hive/warehouse/jardin.db/tweet_api** en même temps il va supprimer les fichiers que nous avons ajouté dans hdfs (requêtage Hadoop). Si on supprime la table, Hive va supprimer aussi les données qui sont dans la table. (Hive Managed TABLE).

La deuxième manière de faire est de créer des tables externes : les fichiers peuvent rester dans hdfs mais Hive va créer une meta-donnée dans notre database, qui exprime juste la structure de notre base. On peut aussi créer des tables qui sont stockées localement. Ils ne sont pas distribués et partitionnés par Hadoop avec la fonction **LOAD DATA LOCAL INPATH** **'/home/hduser1/jardin' OVERWRITE INTO TABLE tweet_api;**

Le Dossier jardin sur HDFS disparaît.

```
hadoop fs -ls /jardin
```

```
15/02/23 23:48:28 WARN util.NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
```

```
ls: `/jardin': No such file or directory
```

Pour savoir où nos bases de données sont stockées, nous utilisons la fonction **describe formatted** de HIVE.

Le résultat de la fonction :

```
describe formatted tweet_api;
```

```
# col_name          data_type      comment
```

```
tweet_json         string
```

```
# Detailed Table Information
```

```
Database:          jardin
```

```
Owner:             hduser1
```

```
CreateTime:        Mon Feb 23 23:44:40 CET 2015
```

```
LastAccessTime:     UNKNOWN
```

```
Protect Mode:       None
```

```
Retention:          0
```

```
/*ici on voit où est ce que la base a été stocké */
```

```
Location:           hdfs://localhost:9000/user/hive/warehouse/jardin.db/tweet_api
```

```
Table Type:         MANAGED_TABLE
```

```
Table Parameters:
```

```
COLUMN_STATS_ACCURATE    true
```

```
numFiles               2
```

```
numRows                 0
```

```
rawDataSize            0
```

```
totalSize               45671682
```

```
transient_lastDdlTime   1424731592
```

```
# Storage Information
```

```
SerDe Library:          org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
```

```
InputFormat:             org.apache.hadoop.mapred.TextInputFormat
```

```
OutputFormat:            org.apache.hadoop.hive ql.io.HiveIgnoreKeyTextOutputFormat
```

```
Compressed:              No
```

```
Num Buckets:              -1
```

```
Bucket Columns:          []
```

```
Sort Columns:             []
```

```
Storage Desc Params:
```

```
serialization.format      1
```

```
Time taken: 0.062 seconds, Fetched: 31 row(s)
```

On peut restructurer les données pour avoir une exploitation correcte et pouvoir changer les formats.

Il aurait été intéressant de construire notre base avec le bon format de chaque variable mais comme notre fichier est de nature JSON avec des variables complexes (Array, Map, Struct), nous n'avons pas réussi à compiler un fichier JAR Serde qui soit à jour avec la version de Hive que nous utilisons.

Le premier fichier JAR que nous avons trouvé a fonctionné sur les fichiers JSON non complexes (<https://code.google.com/p/hive-json-serde/downloads/list>).

Le second JAR, le plus connu, fonctionne sur des structures complexes. Il est prioritairement fait pour Cloudera 4 et 5, il n'a pas fonctionné avec notre version 1.4 de HIVE, (ni là 1.3) (<https://github.com/rcongiu/Hive-JSON-Serde>).

Il nous reste deux moyens pour structurer nos données dans Hive avec la fonction `get_json_object` ou bien `json_tuple` précédé par `lateral view`. Dans la

documentation de Hive il est précisé que `json_tuple` est plus efficient car il retourne un tuple de valeur en une seule fois.

Après avoir récupéré les données variable par variable nous avons remarqué qu'il y a des dates (`created_at`) qui sont **NULL**. Afin de régler le problème, nous avons ajouté un row format en utilisant le fichier JAR SERDE de ColumnarSerDe afin de redécouper les données en colonnes. (RCfile)

1- Création de la table de TWITTER

Voir Annexe 1 pour le code

2- Création de la table Mairie de Paris

Nous avons remarqué qu'il n'était pas possible de créer cette table à l'aide de l'interface graphique.

```
CREATE EXTERNAL TABLE IF NOT EXISTS jardin_mairie (geom_x_y
string, geom string, annee_ouve string, annee_reno string,
annee_chan string, nom_ev_off string, ancien_nom string,
adresse_nu string, adresse string, ardt string, ouvert_fer
string, surface_to string)
row format delimited
fields terminated by '\\;'
lines terminated by '\\n'
tblproperties ("skip.header.line.count"="1");
```

`tblproperties` : Cette option nous permet de supprimer la première ligne de notre table car elle contient les titres des variables.

Chargement de données.

```
LOAD DATA INPATH '/mairie/parcsetjardinsparis2010.csv'
OVERWRITE INTO TABLE jardin_mairie;
```

3- Création de la table Jardin de Paris

```
CREATE TABLE jardin_service (jardin_json STRING);
```

Chargement de données avec la fonction local d'un fichier local.

```
LOAD DATA LOCAL INPATH
'/home/hduser1/base_projet/data_utf8.json' OVERWRITE INTO TABLE
jardin_service;
```

Restructurer les données pour avoir une exploitation correcte et pour pouvoir changer les

formats. On crée la table « jardin_nb » en imposant le format avec des TAB et des retours à la ligne.

```
CREATE TABLE jardin_nb row format delimited fields terminated by
'\t' LINES TERMINATED BY '\n' STORED AS textfile AS select
get_json_object(jardin_service.jardin_json, '$.nom[0]') as nom,
get_json_object(jardin_service.jardin_json, '$.service') as
service, get_json_object(jardin_service.jardin_json,
'$.histoire') as histoire,
get_json_object(jardin_service.jardin_json, '$.Adresse') as
Adresse, get_json_object(jardin_service.jardin_json, '$.link')
as link, get_json_object(jardin_service.jardin_json,
'$.nb_service') as nb_service from jardin_service;
```

Vérification de l'intégrité des données avec un simple select.

```
select nom from jardin_nb;
```

B- Fonction simple

Dans cette partie, nous allons vous présenter la manipulation des données sur les tables que nous venons de créer.

Dans un cas classique non optimisé, l'utilisateur peut lancer une requête en sélectionnant l'ensemble des éléments avec une limite à 100 :

```
select * from tweet_api limit 100;
```

De la même manière que sur du SQL, nous pouvons spécifier des conditions avec Where.

```
select
get_json_object(tweet_api(tweet_json, '$.user.name') as username,
get_json_object(tweet_api(tweet_json, '$.text') as text
from tweet_api
where get_json_object(tweet_api(tweet_json,
'$.user.name')='Cheteteu';
```

La fonction LIKE peut être utilisée de la même manière qu'en SQL.

```
select
get_json_object(tweet_api(tweet_json, '$.user.name') as username,
get_json_object(tweet_api(tweet_json, '$.text') as text
from tweet_api
where get_json_object(tweet_api(tweet_json, '$.user.name') like
'%hete%';
```

La fonction GROUP BY peut être utilisée de la même manière qu'en SQL

```
select user_lang, count(*) as count
from text_tweet
group by user_lang sort by user_lang desc ;
```

La fonction HAVING peut être utilisée de la même manière qu'en SQL

```
select created_at, count(*)
from text_tweet
```

```
Group by created_at
having created_at rlike '.*(Oct|Des|Jan|Feb).*';
```

La fonction GROUP BY inclut aussi des fonctions Sets, Cube, Rollup.

```
SELECT a, b, SUM(c) FROM t1 GROUP BY a, b GROUPING SETS ((a,b), a)
équivalent à :
SELECT a, b, SUM(c) FROM t1 GROUP BY a, b
UNION ALL
SELECT a, NULL, SUM(c) FROM t1 GROUP BY a
```

Group by with Cube

```
SELECT a, b, c, SUM(d) FROM t1 GROUP BY a, b WITH CUBE
équivalent à :
SELECT a, b, c, SUM(d) FROM t1 GROUP BY a, b, c GROUPING SETS
((a,b,c), (a,b), (b,c), (a,c), a,b,c, ())
```

Group by with Rollup

```
SELECT a, b, c, SUM(d) FROM t1 GROUP BY a, b WITH ROLLUP
équivalent à :
SELECT a, b, c, SUM(d) FROM t1 GROUP BY a, b, c GROUPING SETS
((a,b,c), (a,b), a, ())
```

Pour ne pas lister toutes les fonctions qui existent on va les ajouter dans l'annexe 2. On va en utiliser quelques-unes par la suite.

Les requêtes peuvent être lancées par ligne de commande et donc inclues dans des scripts.
Exemple :

```
hive -e 'select a.col from table a'
Ou bien encore avec des scripts SQL pré-construits
hive -i initialize.sql
```

C- Partitionnement

Le partitionnement dans Hive est un outil d'organisation des bases de données. Par exemple : Notre programme Tweepy nous fournit un fichier pour chaque filtre. On pourra classer chacun de ces fichiers dans une même table.

Le partitionnement est un outil de sélection, qui ne coûte rien en stockage.

On pourra l'ajouter dans la fonction Where ou bien dans la fonction Select. Le nombre de partitionnement n'est pas limité.

Exemple :

```
CREATE TABLE tweet (tweet_json STRING)
PARTITIONED BY (source STRING);
```

```
DESCRIBE FORMATTED TWEET;
# Partition Information
# col_name                data_type                comment
```

```
#source                                string

LOAD DATA INPATH '/jardin/twitDB.json' OVERWRITE INTO TABLE tweet
PARTITION(source='paris jardin');
LOAD DATA INPATH '/jardin/twitDB2.json' OVERWRITE INTO TABLE tweet
PARTITION(source='parc jardin');
SELECT * FROM TWEET WHERE source = 'parc jardin' limit 100;
```

D- Multiple Inserts

On peut créer une table de plusieurs échantillons en appliquant un filtre sur plusieurs valeurs. Cela est intéressant pour réduire la taille de grandes tables de données. Exemple : si on prend la table de la Mairie de PARIS on pourra créer 20 tables et chaque table représentera un arrondissement.

Exemple :

On crée deux tables de même nature que notre table de base avec la fonction LIKE

```
CREATE TABLE IF NOT EXISTS jardin_arond_19
LIKE jardin_mairie;
```

```
CREATE TABLE IF NOT EXISTS jardin_arond_20
LIKE jardin_mairie;
```

Puis on crée notre requête de multiple insert.

```
FROM jardin_mairie
INSERT OVERWRITE TABLE jardin_arond_19 SELECT * WHERE ardt = '75019'
INSERT INTO TABLE jardin_arond_20 SELECT * WHERE ardt = '75020';
```

L'`order by` est aussi inclut dans le HiveQL, par défaut, il retourne les valeurs de manière ascendante, mais `order by` est très coûteux car il utilise juste un reduceur pour trier tout la table. Dans notre exemple, nous ordonnons les utilisateurs de manière descendante selon le nombre de retweet.

```
select user_lang, retweeted_retweet_count, retweeted_favorite_count
from text_tweet ORDER BY user_lang ;
```

La fonction `sort by` ne garantit pas de retourner la table toute entière par ordre mais par bocks. Cela nous mène à utiliser plusieurs reduceur (grâce à la fonction shuffle avant les reduceurs) et à profiter du système distribué.

```
select user_lang, retweeted_retweet_count, retweeted_favorite_count
from text_tweet SORT BY user_lang ;
```

La fonction `DISTRIBUTE by` nous permet de contrôler comment distribuer les données sur nos reduceurs, dans ce cas il faut prendre une variable qui partitionne le plus possible notre table en part égale. Mais cette fonction ne permet pas de faire de l'ordre dans la sortie de notre requête.

```
select user_lang, retweeted_retweet_count, retweeted_favorite_count
from text_tweet DISTRIBUTE BY user_lang ;
```

Pour mettre de l'ordre après avoir distribué notre table on peut utiliser la fonction `sort by`. La fonction `cluster by` combine les deux fonctions (Distribute by et sort by) si on utilise la même variable.

```
select user_lang, retweeted_retweet_count, retweeted_favorite_count
from text_tweet cluster BY user_lang ;
```

E- Les Buckets :

Les buckets peuvent être utilisés sur une table ou bien sur une partition pour les décomposer en plusieurs parties. C'est une approche pour distribuer ou classer les tables. Pour créer des échantillons plus efficaces et une meilleure performance des jointures. Nous pourrions ainsi les trier. Voici la requête :

```
CREATE TABLE tweet_text_bucketed(created_at STRING, text STRING)
CLUSTERED BY(created_at) INTO 25 BUCKETS;
```

Ici hive crée une fonction de hachage sur la variable « created_at » et décompose les tables en 25 parties presque égales.

```
set hive.enforce.bucketing = true;
```

Cette commande est obligatoire pour permettre d'utiliser le nombre de reducers égal au nombre de buckets.

```
FROM text_tweet
INSERT OVERWRITE TABLE tweet_text_bucketed
SELECT created_at , text ;
```

Voici une ligne de la sortie de HIVE (On a un seul map !!)

```
Hadoop job information for Stage-1: number of mappers: 1; number of
reducers: 25
```

Buckets Simpling et Block Simpling

Buckets simpling : Création d'échantillons applique sur les tables bucketer.

1. `SELECT * FROM tweet_text_bucketed TABLESAMPLE (BUCKET 3 OUT OF 25 ON created_at) ;`
2. `SELECT * FROM tweet_text_bucketed TABLESAMPLE (BUCKET 3 OUT OF 25 ON rand() ) ;`

Block simpling : Création d'échantillons de n'importe quelle table on utilisant les blocs HDFS. Il est dépendant de la compression du format au moment de création de la base (CombineHiveInputFormat). Il faut faire attention au pourcentage qui est différent du nombre de lignes. Le pourcentage n'est pas une valeur exacte comme les lignes.

3. `SELECT * FROM text_tweet TABLESAMPLE (5 PERCENT) ;`
4. `SELECT * FROM text_tweet TABLESAMPLE (90M) ;`
5. `SELECT * FROM text_tweet TABLESAMPLE (10 ROWS) ;`

F- Streaming :

Avant de passer sur les jointures on va utiliser la fonction TRANSFORM qui découle du principe de Hadoop STREAMING qui permet de personnaliser Hive en utilisant une langue différente. Dans notre exemple on a utilisé le langage PYTHON.

On voudrait nettoyer les noms des jardins, parcs des deux tables (scrapy / Mairie) pour appliquer la jointure et construire une nouvelle table plus riche.

Il faut absolument que les variables qu'on voudrait transformer ne contiennent pas de tabulation.

La syntaxe de la requête transforme :

```
SELECT TRANSFORM (col1 [, col2...coln])
```

//On sélectionne les variables qu'on souhaite

```
USING 'Code File|Program' [AS (list of columns [and casts])]
```

// on utilise notre code puis on indique les variables de sortie

```
FROM SourceTable;
```

On va utiliser deux programmes Python (chaque programme pour une table), dans le but d'éliminer les stop words spécifique à notre problématique.

1-On ajoute le fichier python

```
ADD FILE /home/hduser1/base_projet/clean_jardin.py;
```

2-création de la table

```
Create table jardin_nb AS
```

3-On sélectionne toutes les variables de la tables, et on applique des fonctions hive LCASE pour minuscule, et regexp_replace remplace les tirets, les parenthèses ouvrantes et fermantes, les simples côtes, par des espaces simples pour les deux variables « nom » et « adresse » avant de les transformer.

```
SELECT TRANSFORM (LCASE(regexp_replace(nom, "-|\\'|\\|\\|) |\\|(\"|\" \" ) )
, service, histoire, LCASE(regexp_replace(Adresse, "-|\\'|\\|\\|) |\\|(|\\|\\|
[|\\|\\|)", " " )), link, nb_service)
```

4- on utilise notre fichier clean_jardin.py

```
USING 'clean_jardin.py'
```

5-La projection de no variable.

```
AS nom , service, histoire, Adressej, link, nb_service FROM jardin_nb ;
```

Voici le code Python de notre fichier clean_jardin.py

On a ajouté des stops words qui sont sélectionnés d'après leur fréquence élevée et qui ne donne pas de sens sur le nom : avenue, porte, centre, tep, saint, promenade, place, espace,

vert, jardinet, vegetalises, murs, bp, talus, rue, square, alle, jardins, hotel, jardinieres, placette, medieval, mur, champs, monument, bois, terrain, impasse, école, stade, +, plantée, locaux, gare, zac, bibliotheque, gymnase, rues, pelouse, esplanade, hopital, maison, piscine, arrondissement, immeubles, passage, quai, musee,eglise, plate, bande, boulevard.

```
#!/usr/bin/python
# -*- coding: latin-1 -*-
import os, sys
import re, string
from stop_words import get_stop_words
def supprime_accent(lignee):
    """ supprime les accents du texte source """
    accents = { 'a': ['à', 'ã', 'á', 'â'],
                'e': ['é', 'è', 'ê', 'ë'],
                'i': ['î', 'ï'],
                'u': ['ù', 'ü', 'û'],
                'o': ['ô', 'ö', 'ò', 'ó']
              }
    for (char, accented_chars) in accents.iteritems():
        for accented_char in accented_chars:
            lignee = lignee.replace(accented_char, char)
    return lignee
for line in sys.stdin:
    line = line.strip()
    nom, service, histoire, Adresse, link, nb_service = line.split('\t')
    word = supprime_accent(nom);
    words = word.strip().split()
    words = [w for w in words if not w in get_stop_words('fr')]
    Adresses = Adresse.replace('"', '')
    Adresses = Adresses.strip().split(',')
    Adresses = Adresses[0].translate(None, string.digits +
string.punctuation)
    Adresse = supprime_accent(Adresses);
    Adresses = Adresse.strip().split()
    Adresses = [w for w in Adresses if not w in get_stop_words('fr')]
    Adresse = ' '.join(Adresses)
    nom1 = ' '.join(words)
    print "%s\t%s\t%s\t%s\t%s\t%s" % (nom1, service, histoire, Adresse,
link, nb_service)
```

On fait la même chose pour la table de la mairie

Notre requête HIVE

```
ADD FILE /home/hduser1/base_projet/clean_mairie.py;
Create table jardin_mairie_p AS
SELECT TRANSFORM (geom_x_y, geom, annee_ouve, annee_reno, annee_chan
, LCASE(regexp_replace(nom_ev_off, "-|\\'|\\\\) |\\\"(", " " )) , ancien_nom
, adresse_nu, LCASE(regexp_replace(adresse, "-|\\'|\\\\) |\\\"(", " " )) ,
ardt, ouvert_fer, surface_to)
USING 'clean_mairie.py'
AS geom_x_y, geom, annee_ouve, annee_reno, annee_chan, nom_ev_off
, ancien_nom, adresse_nu, adresse, ardt, ouvert_fer, surface_to FROM
```

```
jardin_mairie_format;
```

Le code Python de notre fichier clean_mairie.py dans l'annexe 3

G- Jointure :

HiveQL nous propose les jointures suivantes :

1-JOIN : Intersection de deux tables.

```
SELECT a.val, b.val FROM a JOIN b ON (a.key = b.key) ;
```

2-LEFT, RIGHT, FULL [OUTER] JOIN

LEFT : Intersection de la table A et B plus la table A

RIGHT : Intersection de la table A et B plus la table B

FULL : A+B

```
SELECT a.val, b.val FROM a LEFT OUTER JOIN b ON (a.key = b.key) JOIN c ON  
(c.key = a.key) ;
```

Dans ce cas on aura A + l'intersection A et B puis un filtre entre le résultat et table C. La requête s'exécute de gauche à droite.

3-LEFT SEMI-JOIN

SEMI-JOIN (contrairement à OUTER-JOIN, une seule ligne est retournée et non pas toutes les lignes égales à une même valeur). Si dans B il y a beaucoup de valeurs égales à une valeur dans A, une seule est retournée.

4-Produit cartésiens ou cross-join

Toutes les colonnes de A sans restriction sont jointes à B.—

```
SELECT a.*, b.* FROM a CROSS JOIN b;
```

5-Map-side Joins

Hive utilise un concept qui s'appelle streaming. Une jointure entre A et B a besoin d'un MAPREDUCE avec le concept de streaming. Le processus streaming se déclenche dans la phase REDUCE sur une Table une fois que la fonction map est prête puis le processus shuffle sur les deux tables.

Un réducteur aura par exemple les clés, valeurs,etc. d'une table puis le processus de streaming s'applique sur l'autre table. Il passe un par un sur les valeurs de l'autre table et lie les valeurs ensemble. Par exemple, dans MONGODB ce principe n'existe pas. La fonction STREAMABLE nous permet de choisir la table à streamer, il faut prendre la grande table pour ne pas saturer la mémoire.

Si on a par exemple trois jointures, il faut toujours commencer par les plus petites.

Voici la requête :

```
SELECT STREAMABLE(a) a.*, b.*, c.*  
FROM a
```

```
LEFT JOIN b ON (a.key = b.key) ;  
JOIN c ON (a.xyz = c.xyz)
```

Pour le concept de MAP-SID JOINS on a une restriction :
Il faut que les tables soit allouables dans la mémoire sauf celle qu'on a choisi de streamer.
Pour les tables qui peuvent être allouées dans la mémoire Hive crée une table avec une fonction hacher.

Voici la requête :

```
SELECT MAPJOIN(b) a.*, b.* FROM a  
JOIN b ON (a.key = b.key) ;
```

Ici la table B sera hacher et la table A sera la table streamée. Dans ce cas on ne pourra pas faire de jointure FULL ou bien Right Outer Joins, ni d'union entre les requêtes.

Une ligne de commande est aussi présente pour que HIVE fasse le boulot à notre place.

```
set hive.auto.convert.join=true  
SELECT a.*, b.* FROM a  
JOIN b ON (a.key = b.key) ;
```

Revenons à nos tables. Comme on est bien fainéants et qu'on voudrait faire notre fameuse jointure, on ajoute le scripte pour l'optimisation de la jointure.

```
set hive.auto.convert.join=true
```

1ere jointure avec nos tables brutes.

```
SELECT nom_ev_off, nom, service  
FROM jardin_mairie JOIN jardin_nb ON nom_ev_off = nom;  
Résultat : 1 réponse
```

```
Square Louis XVI      Square Louis XVI      ["Wi-fi", "Points  
d'eau", "Bacs à sable"]  
Time taken: 27.212 seconds, Fetched: 1 row(s)
```

Le problème que nous rencontrons ici est lié à la qualité de données hétérogènes.
2ème jointure après le nettoyage des noms, des jardins, etc.

```
SELECT nom_ev_off, nom, ouvert_fer  
FROM jardin_mairie_p LEFT SEMI JOIN jardin_nb_p ON nom_ev_off = nom;  
  
Time taken: 20.411 seconds, Fetched: 95 row(s)
```

On arrive à avoir une jointure à 80% que l'on aimerait encore améliorer. Il nous reste 23.
C'est pour cela qu'on avait prévu de nettoyer aussi bien les adresses des jardins.

Si on fait la jointure avec les noms et les adresses :

```
SELECT nom_ev_off, nom, ouvert_fer  
FROM jardin_nb_p LEFT SEMI JOIN jardin_mairie_p ON nom_ev_off = nom and  
adressej = adresse;  
Time taken: 20.073 seconds, Fetched: 55 row(s)
```

Puis une autre avec juste l'adresse :

```
SELECT nom_ev_off, nom, ouvert_fer
FROM jardin_nb_p LEFT SEMI JOIN jardin_mairie_p ON adressej = adresse;
Time taken: 21.754 seconds, Fetched: 76 row(s)
```

Pour construire notre base enrichie, on fait l'union de la jointure nom et adresse puis on crée une table avec la fonction distincte sur les noms par exemple.

```
CREATE TABLE join_NAME as
SELECT *
FROM jardin_nb_p LEFT SEMI JOIN jardin_mairie_p ON nom_ev_off = nom;
CREATE TABLE join_add as

SELECT *
FROM jardin_nb_p LEFT SEMI JOIN jardin_mairie_p ON adressej = adresse;
CREATE TABLE join_uni as
SELECT *
FROM join_NAME
UNION ALL
SELECT *
FROM join_add;
Create table final as
select distinct(nom_ev_off), nom, service, histoire, Adressej, link,
nb_service, geom_x_y, geom, annee_ouve, annee_reno, annee_chan
, ancien_nom, adresse_nu, adresse, ardt, ouvert_fer, surface_to from
join_uni;
```

Cette table finale peut être une candidate pour visualiser notre projet sur une carte car on a soigneusement pris toutes les variables de chacune des tables.

Les mots clés que nous avons supprimés peuvent être importants pour les adresses. Exemple : « rue de Belleville » et « boulevard de Belleville ». Si on surprime « rue » et « boulevard », il reste bien Belleville ...On peut aussi construire un stop words simple juste pour les adresses.

Par contre, si on examine le reste de la jointure sur les Noms, on trouve des Noms nouveaux et des Noms anciens dans la même variable. Exemple : (["Square Danielle-Mitterrand (ex-Jardin de la rue de Bièvre)"]). C'est possible de continuer à éliminer avec des stops word HAND-MADE et au final on trouve 105 /118. Nous avons vérifié ce chiffre avec l'outil DSS de la boîte DATAIKU avec la méthode Fuzzy join.

« 'Fuzzy' means that the join can match even if the two strings being matched are not exactly equal, but close. »

Si on se met dans le cas d'une grande base de données, il faut envisager d'utiliser d'autres méthodes comme la jointure par similarité (cette partie sera reprise pour la suite du projet dans le UE RCP216 pour comparaison).

Statistiques et Data Mining (text mining)

L'idée est de pouvoir préparer une analyse d'informations concernant le comptage des termes, le poids des termes, la suppression des « stop words », ainsi que le travail sur des

paires de mots.

Les fonctions qu'on va utiliser :

H- Explode :

Elle prend un tableau en entrées et elle le décompose par ligne. Cette fonction nous permet de faire un comptage de mots très rapidement.

Exemple :

```
CREATE TABLE word_counts AS
SELECT word, count(1) AS count FROM
(SELECT explode(split(text, ' ')) AS word FROM text_tweet) word
GROUP BY word
ORDER BY word;
select * from word_counts where count>30 ORDER BY count desc limit 100;
```

Résultat de notre requête

Parc 4898	Princes 1479	Si 977	#Paris 672	pour 524
RT 4854	! 1421	Le 944	Chelsea 668	di 502
de 4842	en 1331	@J_AdoreParis: 845	et 663	est 497
Paris 4463	jardin 1315	RT 845	PARIS 632	htt... 474
des 3985	: 1300	Jardin 819	ce 626	#PSGFCGB 462
au 3279	at 1264	Restaurant 810	soir 571	COME 460
du 2419	Saint-Germain 1247	the 784	les 571	@ 457
le 2041	a 1233	760	in 565	Buttes 455
à 1982	parc 1117	@PSG inside: 694	un 531	te 447
- 1780	la 999		528	21h 447

Nous pouvons remarquer que le nom « parc » est plus utilisé que le mot jardin (presque 3 fois plus). Les mots en rose sont associés aux parcs ou jardins tandis que les mots en bleu sont davantage liés au foot à cause de « parc des princes ».

Il nous faudrait obligatoirement un passage par la fonction « transformation » pour supprimer les STOPT WORDS dans plusieurs langues.

Dans notre sortie, on peut aussi voir des liens HTML qui polluent notre analyse. Exemple :

<http://t.co/IVQxWD2rZz> 440
<http://t.co/GhTTDsXFnr> 433

On va passer notre variable textes de twitter pour supprimer des stop-words , les caractères spéciaux, ainsi que les liens (ANNEX 4 pour le code).

paris 9820	... 802	di 503	21h 447	una 407
rt 5815	psginside 747	ville 503	foraine 444	hui 407
princes 2369	tuileries 686	meet 498	gustaría 440	desde 407
germain 1684	c 624	france 481	escaparte 440	vistas 407
psg 1191	2 610	come 479	este 440	aquí 406
chelsea 1177	relais 603	htt... 474	medio 440	pasarías 405
jadoreparis 845	soir 594	psgfcgb 463	paraíso 440	tarde 405
restaurant 811	caen 572	belleville 458	vs 439	impresionantes 405
	bordeaux 535	el 455	16eme 422	1 400
	chaumont 526	fete 450	aujourd 409	luckybsmith 386
	buttes 525		o 408	

Nous pouvons conclure que notre liste de stop words est incomplète. Mais nous allons continuer notre analyse de texte en utilisant d'autres fonctions de HIVE.

I- Sentence et Ngrams

Sentence : Découpe notre string et le transforme en tableau de mots.

Ngrams : Retourne les N mots d'un string les plus fréquents.

Par exemple : cette requête retourne les 50 paires de mots les plus fréquents.

```
ngrams (sentences (lower (text)) , 2 , 50)
```

Dans notre cas : on va l'utiliser avant et après notre transformation de texte pour comparaison.

Avant la fonction transform	Après la fonction transform
<pre>{ "ngram": ["http", "t.co"], "estfrequency": 7353.0 } { "ngram": ["parc", "des"], "estfrequency": 3118.0 } { "ngram": ["au", "parc"], "estfrequency": 2594.0 } { "ngram": ["des", "princes"], "estfrequency": 2207.0 } { "ngram": ["paris", "saint-germain"], "estfrequency": 1278.0 } { "ngram": ["paris", "http"], "estfrequency": 1041.0 } { "ngram": ["de", "paris"], "estfrequency": 971.0 } { "ngram": ["du", "parc"], "estfrequency": 965.0 } { "ngram": ["jardin", "des"], "estfrequency": 870.0 } { "ngram": ["paris", "rt"], "estfrequency": 850.0 } { "ngram": ["rt", "j_adoreparis"], "estfrequency": 844.0 } { "ngram": ["rt", "si"], "estfrequency": 844.0 } { "ngram": ["parc", "de"], "estfrequency": 840.0 } { "ngram": ["rt", "psg_inside"], "estfrequency": 694.0 } { "ngram": ["des", "tuileries"], "estfrequency": 610.0 } { "ngram": ["du", "jardin"], "estfrequency": 602.0 } { "ngram": ["relais", "du"], "estfrequency": 601.0 } { "ngram": ["ce", "soir"], "estfrequency": 562.0 } { "ngram": ["at", "the"], "estfrequency": 552.0 } { "ngram": ["le", "jardin"], "estfrequency": 547.0 } { "ngram": ["jardin", "de"], "estfrequency": 538.0 } { "ngram": ["meet", "me"], "estfrequency": 493.0 } { "ngram": ["http", "t"], "estfrequency": 488.0 } { "ngram": ["jardin", "du"], "estfrequency": 466.0 } { "ngram": ["buttes", "chaumont"], "estfrequency": 455.0 }</pre>	<pre>{ "ngram": ["paris", "germain"], "estfrequency": 1651.0 } { "ngram": ["rt", "jadoreparis"], "estfrequency": 844.0 } { "ngram": ["rt", "psginside"], "estfrequency": 694.0 } { "ngram": ["buttes", "chaumont"], "estfrequency": 525.0 } { "ngram": ["chaumont", "paris"], "estfrequency": 452.0 } { "ngram": ["paris", "come"], "estfrequency": 447.0 } { "ngram": ["princes", "psgfcgb"], "estfrequency": 446.0 } { "ngram": ["psginside", "paris"], "estfrequency": 445.0 } { "ngram": ["21h", "princes"], "estfrequency": 444.0 } { "ngram": ["fete", "foraine"], "estfrequency": 444.0 } { "ngram": ["germain", "bordeaux"], "estfrequency": 443.0 } { "ngram": ["soir", "21h"], "estfrequency": 443.0 } { "ngram": ["bordeaux", "soir"], "estfrequency": 442.0 } { "ngram": ["come", "meet"], "estfrequency": 442.0 } { "ngram": ["meet", "fete"], "estfrequency": 442.0 } { "ngram": ["luckybsmith", "paris"], "estfrequency": 440.0 } { "ngram": ["escaparte", "este"], "estfrequency": 439.0 } { "ngram": ["este", "paraíso"], "estfrequency": 439.0 } { "ngram": ["gustaria", "escaparte"], "estfrequency": 439.0 } { "ngram": ["jadoreparis", "buttes"], "estfrequency": 439.0 } { "ngram": ["medio", "paris"], "estfrequency": 439.0 } { "ngram": ["paraíso", "medio"], "estfrequency": 439.0 } { "ngram": ["rt", "gustaria"], "estfrequency": 439.0 } { "ngram": ["belleville", "paris"], "estfrequency": 410.0 } { "ngram": ["aujourd", "hui"], "estfrequency": 406.0 }</pre>

On peut remarquer que « parc des princes » est très fréquent et qu'il faut l'éliminer de notre base pour faire une bonne prévision.

Plusieurs méthode de classification sur les mots simples, ou groupes de mots peuvent nous aider à mieux comprendre nos données, ainsi qu'une meilleure performance de calcul par la suite car on pourra travailler sur une partie des données seulement.

Dans cette analyse ne prend en considération qu'une seule variable qui est le texte, mais on pourra prendre en considération le temps, le lieux, les utilisateurs, etc. On n'a pas encore cette possibilité avec Hive, sauf avec la fonction where, grâce à laquelle on pourra construire de petits échantillons qu'on analysera avec des programmes statistiques comme R. (RHIVE).

J- Context ngrams

Elle permet une sélection d'un nombre de mots classés par fréquence autour des mots qu'on définit dans un tableau.

Par exemple nous aimerions voir les mots qui existent autour du premier Ngram le plus fréquent :

Résultat de la requête

```
{ "ngram": ["bordeaux"], "estfrequency": 443.0 }
{ "ngram": ["reçoit"], "estfrequency": 268.0 }
{ "ngram": ["vs"], "estfrequency": 216.0 }
{ "ngram": ["psg"], "estfrequency": 197.0 }
{ "ngram": ["akan"], "estfrequency": 116.0 }
{ "ngram": ["2"], "estfrequency": 56.0 }
{ "ngram": ["gagal"], "estfrequency": 45.0 }
```

Requête

```
SELECT
explode(context_ngrams(sentences(lower(text)),
array("paris", "germain", null), 100))
AS ngs FROM text_stopw;
```

<pre>{ "ngram": ["0"], "estfrequency": 36.0 } { "ngram": ["3"], "estfrequency": 21.0 } { "ngram": ["y"], "estfrequency": 21.0 } { "ngram": ["chelsea"], "estfrequency": 17.0 } { "ngram": ["x"], "estfrequency": 16.0 } { "ngram": ["kontra"], "estfrequency": 12.0 } { "ngram": ["princes"], "estfrequency": 12.0 } { "ngram": ["recevoir"], "estfrequency": 10.0 } { "ngram": ["1"], "estfrequency": 8.0 } { "ngram": ["accueille"], "estfrequency": 8.0 } { "ngram": ["berhadapan"], "estfrequency": 8.0 } { "ngram": ["affluence"], "estfrequency": 6.0 } { "ngram": ["fc"], "estfrequency": 6.0 } { "ngram": ["football"], "estfrequency": 5.0 }</pre>	On peut confirmer que notre premier NGRAM est associer au foot.
---	---

Nous allons prendre 3 clés de mots qui nous intéressent.

<pre>SELECT explode(context_ngrams(sentences(lower(text)), array("buttes", "chaumont", null), 100)) AS ngs FROM text_stopw;</pre>	<pre>SELECT explode(context_ngrams(sentences(lower(text)), array("fete", "foraine", null), 100)) AS ngs FROM text_stopw;</pre>	<pre>sELECT explode(context_ngrams(sentences(lower(text)), array("belleville", "paris", null), 100)) AS ngs FROM text_stopw;</pre>
<pre>{ "ngram": ["paris"], "estfrequency": 452.0 } { "ngram": ["renovation"], "estfrequency": 46.0 } { "ngram": ["book"], "estfrequency": 3.0 } { "ngram": ["parijs"], "estfrequency": 3.0 } { "ngram": ["poursuit"], "estfrequency": 2.0 } { "ngram": ["cold"], "estfrequency": 1.0 } { "ngram": ["jerome"], "estfrequency": 1.0 } { "ngram": ["photos"], "estfrequency": 1.0 } { "ngram": ["reference"], "estfrequency": 1.0 }</pre>	<pre>{ "ngram": ["tuileries"], "estfrequency": 393.0 } { "ngram": ["tui"], "estfrequency": 49.0 } { "ngram": ["winter"], "estfrequency": 2.0 }</pre>	<pre>{ "ngram": ["france"], "estfrequency": 2.0 } { "ngram": ["belleville"], "estfrequency": 1.0 } { "ngram": ["ile"], "estfrequency": 1.0 }</pre>

Le premier nous indique qu'il y a des rénovations au parc des buttes Chaumont, le deuxième qu'au tuileries il y a une fête foraine et le troisième pas grand chose (la fréquence des mots est très petite mais si non il y a une belle vue à Paris).

Conclusion

Malheureusement, Hive n'est pas prévu pour les traitements transactionnels en ligne et n'offre pas des performances adaptées aux requêtes en temps réel ou encore aux mises à jour ligne par ligne (un des principes de l'entrepôt de données).

Hive possède beaucoup de fonctions plus développées que SQL car ils sont basés sur le langage Java. Dans une architecture distribuée, elle est munie des méthodes de buketing et de partitionnement pour créer des échantillons et optimiser les JOB MAPREDUCE et éviter les saturations du réseau.

Pour optimiser le traitement des données, des formats de fichiers ont été développés pour améliorer le temps de traitement, de stockage et de chargements des données massives.

Les jointures dans Hive sont optimisées pour les traitements de grandes bases de données mais ne permettent pas encore faire des jointures par similarité. C'était une solution que l'on a souhaité tester sur nos bases de données.

Pour les formats de données, Hive est muni de plusieurs fonctions pour traiter les fichiers json, csv, xpath pour les formats xml mais il est pauvre en ce qui concerne les données géographiques. On peut trouver des solutions pour Hadoop qu'on pourra traiter avec Hive.

Pour la partie « analyses statistiques », les fonctions sont très pauvres pour le moment sauf pour des études descriptives, ou la création d'échantillons. Depuis septembre 2014, on peut connecter HIVE et le logiciel R grâce à Hive server. Pour continuer nos analyses statistiques (STAT211), on pourra utiliser RHIVE (<https://github.com/nexr/RHive>).

Notre seul regret est de ne pas avoir pu travailler sur des échantillons de données suffisamment volumineux pour en tirer des conclusions plus constructives sur le temps des requête.

Bibliographie

1. <http://fr.wikipedia.org/wiki/Hadoop>
2. http://en.wikipedia.org/wiki/Apache_Hive
3. <http://it-ebooks.info/book/941/>
4. <https://cwiki.apache.org/confluence/display/Hive/Home>
5. <http://hadoop.apache.org/>

ANNEX 1

```
CREATE TABLE text_tweet ROW FORMAT SERDE "org.apache.hadoop.hive.serde2.columnar.ColumnarSerDe"
  STORED AS RCFile
AS select v.created_at, v.text, v.id, v.id_str, v.source, v.rel, v.truncated, v.in_reply_to_status_id, v.in_reply_to_status_id_str,
v.in_reply_to_user_id, v.in_reply_to_user_id_str, v.in_reply_to_screen_name, v.user, v.geo, v.coordinates, v.contributors,
v.retweet_count, v.favorite_count, v.entities,
v.place, v.favorited, v.retweeted, v.possibly_sensitive, v.filter_level, v.lang, v.timestamp_ms, v.retweeted_status,

v1.user_id, v1.user_id_str, v1.name, v1.screen_name, v1.location, v1.url, v1.description, v1.protected, v1.verified,

v1.followers_count, v1.friends_count, v1.listed_count, v1.favourites_count, v1.statuses_count, v1.user_created_at, v1.utc_offset,
v1.time_zone, v1.geo_enabled, v1.user_lang, v1.contributors_enabled, v1.is_translator, v1.profile_background_color,
v1.profile_background_image_url, v1.profile_background_image_url_https,
v1.profile_background_tile, v1.profile_link_color, v1.profile_sidebar_border_color, v1.profile_sidebar_fill_color, v1.profile_text_color,
v1.profile_use_background_image, v1.profile_image_url, v1.profile_image_url_https, v1.profile_banner_url, v1.default_profile,
v1.default_profile_image, v1.follow_request_sent, v1.notifications, v1.user_following, v2.geo_type,
v2.geo_coordinates, v3.type_coordinates, v3.coordinates_coordinates, v4.place_id, v4.place_url, v4.place_place_type, v4.place_name,
v4.place_full_name, v4.place_country_code, v4.place_country, v4.place_bounding_box, v4.place_attributes, v5.entities_hashtags,
v5.entities_trends, v5.entities_urls, v5.entities_user_mentions, v5.entities_symbols, v6.retweeted_retweet_count,
v6.retweeted_favorite_count
from tweet_api jt
LATERAL VIEW json_tuple(jt.tweet_json, 'created_at', 'text', 'id', 'id_str', 'source', 'rel', 'truncated', 'in_reply_to_status_id',
'in_reply_to_status_id_str', 'in_reply_to_user_id', 'in_reply_to_user_id_str', 'in_reply_to_screen_name', 'user', 'geo', 'coordinates',
'place', 'contributors', 'retweet_count',
'favorite_count', 'entities', 'favorited', 'retweeted', 'possibly_sensitive', 'filter_level', 'lang', 'timestamp_ms', 'retweeted_status') v
  as created_at, text, id, id_str, source, rel, truncated, in_reply_to_status_id, in_reply_to_status_id_str, in_reply_to_user_id,
in_reply_to_user_id_str, in_reply_to_screen_name, user, geo, coordinates, place, contributors, retweet_count,
favorite_count, entities, favorited, retweeted, possibly_sensitive, filter_level, lang, timestamp_ms, retweeted_status
LATERAL VIEW json_tuple(v.user, 'id', 'id_str', 'name', 'screen_name', 'location', 'url', 'description', 'protected', 'verified',
'followers_count', 'friends_count', 'listed_count', 'favourites_count', 'statuses_count', 'created_at', 'utc_offset', 'time_zone',
'geo_enabled', 'lang', 'contributors_enabled', 'is_translator', 'profile_background_color', 'profile_background_image_url',
'profile_background_image_url_https', 'profile_background_tile', 'profile_link_color', 'profile_sidebar_border_color',
'profile_sidebar_fill_color', 'profile_text_color', 'profile_use_background_image', 'profile_image_url', 'profile_image_url_https',
'profile_banner_url', 'default_profile', 'default_profile_image', 'follow_request_sent', 'notifications', 'following') v1
  as user_id, user_id_str, name, screen_name, location, url, description, protected, verified, followers_count, friends_count,
listed_count, favourites_count, statuses_count, user_created_at, utc_offset, time_zone, geo_enabled, user_lang,
contributors_enabled, is_translator, profile_background_color, profile_background_image_url,
profile_background_image_url_https, profile_background_tile, profile_link_color, profile_sidebar_border_color,
profile_sidebar_fill_color, profile_text_color, profile_use_background_image, profile_image_url, profile_image_url_https,
profile_banner_url, default_profile, default_profile_image, follow_request_sent, notifications, user_following
LATERAL VIEW json_tuple(v.geo, 'type', 'coordinates') v2
  as geo_type, geo_coordinates
LATERAL VIEW json_tuple(v.coordinates, 'type', 'coordinates') v3
  as type_coordinates, coordinates_coordinates
LATERAL VIEW json_tuple(v.place, 'id', 'url', 'place_type', 'name', 'full_name', 'country_code', 'country', 'bounding_box', 'attributes') v4
  as place_id, place_url, place_place_type, place_name, place_full_name, place_country_code, place_country,
place_bounding_box, place_attributes
LATERAL VIEW json_tuple(v.entities, 'hashtags', 'trends', 'urls', 'user_mentions', 'symbols') v5
  as entities_hashtags, entities_trends, entities_urls, entities_user_mentions, entities_symbols
LATERAL VIEW json_tuple(v.retweeted_status, 'retweet_count', 'favorite_count') v6
  as retweeted_retweet_count, retweeted_favorite_count;
```

ANNEX 2

Mathematical

```
SELECT rand(), a FROM t1; SELECT rand(3), rand(a) FROM t1;
SELECT pow(a, b) FROM t2; SELECT tan(a) FROM t3;
abs(double a)
round(double a, int d)
floor(double a)
```

Collection

```
size(Map<K.V>)
map_keys(Map<K.V>)
map_values(Map<K.V>)
SELECT array_contains(a, 'test') FROM t1;
```

Date

```
unix_timestamp()
year(string d), month(string d), day(string d), hour, second
datediff(string enddate, string startdate)
date_add(string startdate, int days)
date_sub(string startdate, int days)
to_date(string timestamp)
```

Conditional

```
SELECT IF(a = b, 'true result', 'false result') FROM t1;
SELECT COALESCE(a, b, c) FROM t1;
SELECT CASE a WHEN 123 THEN 'first' WHEN 456 THEN 'second'
ELSE 'none' END FROM t1;
SELECT CASE WHEN a = 13 THEN c ELSE d END FROM t1;
```

String

```
SELECT concat(a, b) FROM t1; SELECT concat_ws(sep, a, b) FROM t1;
SELECT regex_replace("Hive Rocks", "ive", "adoop") FROM dummy;
substr(string|binary A, int start)
substring(string|binary A, int start, int length)
sentences(string str, string lang, string locale)
SELECT sentences("Loving this course! Hive is awesome.") FROM dummy;
```

Built-in Aggregate Functions (UDAFs)

```
COUNT(*), COUNT(expr), COUNT(DISTINCT expr)
SUM(col), SUM(DISTINCT col)
AVG, MIN, MAX, VARIANCE, STDDEV_POP
HISTOGRAM_NUMERIC(col, b)
returns array<struct { 'x', 'y' }>
```

ANNEX 3

```
#!/usr/bin/python
# -*- coding: latin-1 -*-
import os, sys
import re
from stop_words import get_stop_words
def supprime_accent(lignee):
    """ supprime les accents du texte source """
    accents = { 'a': ['à', 'ã', 'á', 'â'],
                'e': ['é', 'è', 'ê', 'ë'],
                'i': ['î', 'ï'],
                'u': ['ù', 'ü', 'û'],
                'o': ['ô', 'ö', 'ò', 'ó']
    }
    for (char, accented_chars) in accents.iteritems():
        for accented_char in accented_chars:
            lignee = lignee.replace(accented_char, char)
    return lignee
for line in sys.stdin:
    line = line.strip()
    geom_x_y, geom, annee_ouve, annee_reno, annee_chan, word, ancien_nom,
    adresse_nu, adresse, ardt, ouvert_fer, surface_to = line.split('\t')
    word = supprime_accent(word);
    words = word.strip().split()
    words = [w for w in words if not w in get_stop_words('fr')]
    adresse = supprime_accent(adresse);
    adresse = adresse.strip().split()
    adresse = [w for w in adresse if not w in get_stop_words('fr')]
    adresse = ' '.join(adresse)
    name = ' '.join(words)
    print "%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s" % (geom_x_y,
    geom, annee_ouve, annee_reno, annee_chan, name, ancien_nom,
    adresse_nu, adresse, ardt, ouvert_fer, surface_to)
```

ANNEX 4

```
ADD FILE /home/hduser1/base_projet/top.py;
Create table text_stopw AS
SELECT TRANSFORM (LCASE(text))
USING 'top.py'
AS text
FROM text_tweet ;
#!/usr/bin/python
# -*- coding: latin-1 -*-
import os, sys
import re
import time, string
from stop_words import get_stop_words
def supprime_accent(lignee):
    """ supprime les accents du texte source """
    accents = { 'a': ['à', 'ã', 'á', 'â'],
                'e': ['é', 'è', 'ê', 'ë'],
                'i': ['î', 'ï'],
                'u': ['ù', 'ü', 'û'],
                'o': ['ô', 'ö', 'ò', 'ó'] }
    for (char, accented_chars) in accents.iteritems():
        for accented_char in accented_chars:
            lignee = lignee.replace(accented_char, char)
    return lignee
for line in sys.stdin:
    word = line.strip()
    word = re.sub(r'\w+:\/\/{2}[\d\w-]+(\.[\d\w-]+)*(?:\:\/\/[^\s\/]*)*)*',
    '', word) # supprimer les liens
    word = supprime_accent(word); #supprimer les accents
    word = word.replace('"', ' ')
    word = word.replace('-', ' ')
    word = word.replace('#', ' ')
    word = word.replace("'", ' ')
    word = word.translate(string.maketrans("", ""), string.punctuation)
#supprimer les ponctuations
    words = word.strip().split()
    words = [w for w in words if not w in get_stop_words('tw')] #notre stop
words de twitter quand à créer dans le module stop_words
    text = ' '.join(words)
    print '%s' % (text)
drop Table word_counts;
CREATE TABLE word_counts AS
SELECT word, count(1) AS count FROM
(SELECT explode(split(text, ' ')) AS word FROM text_stopw) word
GROUP BY word
ORDER BY word;
select * from word_counts where count>30 ORDER BY count desc limit 100;
```